

Digital Logic Design — Assignment Solutions

Course Code: CSE 211 | Department: CSE | Metropolitan University

Question Index & Topic Map

Q1	Largest 16-bit binary number → Decimal & Hex	Topic: Number System
Q2	16's complement of $(4793)_{16}$, then to binary & octal	Topic: Complements + Conversions
Q3	Subtract $(27)_8$ from $(22)_{16}$ using 2's complement	Topic: Subtraction Using Complement
Q4	$F = xy + x'z$ to product of maxterms	Topic: Boolean Algebra + POS
Q5	Simplify $((\bar{B}\bar{C}\bar{D} + ABC)D) + \bar{A}D$	Topic: Boolean Simplification
Q6	Change OR ↔ AND in given circuit + write expression	Topic: Logic Diagram
Q7	$F(A,B,C,D) = ABC + A'B + AD'$ as product of maxterms	Topic: SOP to POS Conversion
Q8	Home security system Boolean function $P(A,C,B)$	Topic: Combinational Logic Design
Q9	K-map: $\Sigma m(0,1,3,5,6,7,12,14) + d(2,8,11,15)$	Topic: K-Map Simplification
Q10	Working of all flip-flops with figures and tables	Topic: Sequential Logic / Flip-Flops
Q12	Full Subtractor using Half Subtractors	Topic: Combinational Logic Circuits
Q13	4-input Priority Encoder ($D1 > D0 > D2 > D3$)	Topic: Priority Encoder

Question 1 — Largest 16-bit Binary Number

 **Topic: Number System (Binary, Decimal, Hexadecimal)**

Q1. What is the largest binary number that can be expressed with 16 bits? What are the equivalent decimal and hexadecimal numbers? Use necessary conversion methods.

Step 1: Find the Largest 16-bit Binary Number

With **n bits**, the largest unsigned number is when all bits are 1.

For 16 bits: all 16 bits = 1.

Largest binary = 1111 1111 1111 1111 (16 ones)

Step 2: Convert to Decimal

The largest unsigned n-bit number = $2^n - 1$.

Decimal = $2^{16} - 1 = 65536 - 1 = 65535$

Verification (positional method):

$$\begin{aligned} &= 1 \times 2^{15} + 1 \times 2^{14} + 1 \times 2^{13} + 1 \times 2^{12} + 1 \times 2^{11} + 1 \times 2^{10} + 1 \times 2^9 + 1 \times 2^8 \\ &+ 1 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\ &= 32768 + 16384 + 8192 + 4096 + 2048 + 1024 + 512 + 256 \\ &+ 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 \\ &= 65535 \end{aligned}$$

Step 3: Convert to Hexadecimal

Group binary digits into sets of 4 from the right:

Binary	1111	1111	1111	1111
Hex	F	F	F	F

Hexadecimal = $(FFFF)_{16}$

Verification (decimal to hex):

Step	Quotient	Remainder
$65535 \div 16$	4095	15 $\rightarrow$ F
$4095 \div 16$	255	15 $\rightarrow$ F
$255 \div 16$	15	15 $\rightarrow$ F

$15 \div 16$	0	$15 \rightarrow F$
--------------	---	--------------------

Reading bottom-up: **FFFF** ✓

 Final Answer

Number System	Value
Binary (16-bit)	1111 1111 1111 1111
Decimal	65535
Hexadecimal	FFFF

Question 2 — 16's Complement and Conversions

 **Topic: r's Complement + Hex-to-Binary-to-Octal Conversion**

Q2(a). Find the 16's complement of $(4793)_{16}$.

Concept

For base r , **r's complement** = $(r-1)$'s complement + 1.

For hexadecimal: **16's complement** = (15's complement) + 1 = $(FFFF - N) + 1$.

Step 1: Find 15's complement of 4793

Subtract each digit from 15 (F):

F	F	F	F
-4	-7	-9	-3
B	8	6	C

15's complement = $(B86C)_{16}$

Step 2: Add 1 to get 16's complement

```
B86C
+   1
-----
B86D
```

16's complement of $(4793)_{16} = (B86D)_{16}$

Q2(b). Convert the result of (a) to a binary number.

Hexadecimal to Binary Conversion

Each hex digit converts to a 4-bit binary group:

Hex	B	8	6	D
Decimal	11	8	6	13
Binary	1011	1000	0110	1101

$$(B86D)_{16} = (1011\ 1000\ 0110\ 1101)_2$$

Q2(c). Convert the result of (b) into an octal number.

Binary to Octal Conversion

Group the binary number into sets of 3 bits, starting from the right:

Binary = 1011100001101101 (16 bits)

Group into 3 bits from the right (pad MSB with zeros if needed):

001 011 100 001 101 101

3-bit group	001	011	100	001	101	101
Octal	1	3	4	1	5	5

$$(1011\ 1000\ 0110\ 1101)_2 = (134155)_8$$

Summary of all parts

System	Value
Hex (16's complement)	B86D
Binary	1011 1000 0110 1101
Octal	134155

Question 3 — Subtraction Using 2's Complement

Topic: Subtraction Using Complement

Q3. Subtract $(27)_8$ from $(22)_{16}$ using 2's complement and verify the result in the decimal system.

Step 1: Convert both numbers to Decimal (for verification)

$$(22)_{16} = 2 \times 16^1 + 2 \times 16^0 = 32 + 2 = 34$$

$$(27)_8 = 2 \times 8^1 + 7 \times 8^0 = 16 + 7 = 23$$

So we are computing: $34 - 23 = 11$

Step 2: Convert both to Binary (8 bits)

$$(22)_{16} \rightarrow \text{Binary: } 2=0010, 2=0010 \rightarrow 0010\ 0010$$

$$(27)_8 \rightarrow \text{Binary: } 2=010, 7=111 \rightarrow 010\ 111 \rightarrow \text{padded} \rightarrow 0001\ 0111$$

Number	Decimal	Binary (8-bit)
Minuend $(22)_{16}$	34	0010 0010
Subtrahend $(27)_8$	23	0001 0111

Step 3: Find 2's Complement of the Subtrahend

1's complement of 0001 0111 = flip every bit = 1110 1000

2's complement = 1's + 1 = 1110 1000 + 1 = 1110 1001

Step 4: Add Minuend to 2's Complement of Subtrahend

```
  0010 0010    (34)
+ 1110 1001    (2's comp of 23)
-----
 1 0000 1011
  ↑
End carry → discard
```

Step 5: Check the Carry

An end carry was generated → **result is positive**. Discard the carry.

Result = **0000 1011**₂

Step 6: Convert Result back to Decimal

$$0000\ 1011 = 8 + 0 + 2 + 1 = \mathbf{11}$$

$$\mathbf{Final\ Result = (00001011)_2 = (11)_{10}}$$

Verification

$$(22)_{16} - (27)_8 = 34 - 23 = \mathbf{11} \checkmark$$

Question 4 — Convert to Product of Maxterms

 **Topic: Boolean Algebra + Canonical POS Form**

Q4. Express the Boolean function $F = xy + x'z$ in product of maxterms form. First convert the function using the distributive law and necessary Boolean postulates.

Step 1: Convert SOP to Standard POS Using Distributive Law

$$F = xy + x'z$$

Apply distributive law: $A + BC = (A + B)(A + C)$

$$\begin{aligned} F &= (xy + x') \cdot (xy + z) \\ &= (x' + xy)(z + xy) \\ &= [(x' + x)(x' + y)] \cdot [(z + x)(z + y)] \\ &= [1 \cdot (x' + y)] \cdot [(x + z)(y + z)] \\ &= (x' + y)(x + z)(y + z) \end{aligned}$$

$$F = (x' + y)(x + z)(y + z)$$

Step 2: Expand each term to include all variables (x, y, z)

Term 1: $(x' + y)$ — missing z

$$= x' + y + zz' = (x' + y + z)(x' + y + z')$$

Term 2: $(x + z)$ — missing y

$$= x + z + yy' = (x + y + z)(x + y' + z)$$

Term 3: $(y + z)$ — missing x

$$= y + z + xx' = (x + y + z)(x' + y + z)$$

Step 3: Combine all terms

$$F = (x' + y + z)(x' + y + z')(x + y + z)(x + y' + z)(x + y + z)(x' + y + z)$$

Remove duplicates:

- $(x + y + z)$ — appears twice → keep once
- $(x' + y + z)$ — appears twice → keep once

$$F = (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z')$$

Step 4: Identify Maxterms

Maxterm	x y z	Designation
$(x + y + z)$	0 0 0	M_0
$(x + y' + z)$	0 1 0	M_2
$(x' + y + z)$	1 0 0	M_4
$(x' + y + z')$	1 0 1	M_5

$F(x, y, z) = \Pi M(0, 2, 4, 5)$

Verification using Truth Table

x	y	z	xy	x'z	F
0	0	0	0	0	$0 \rightarrow M_0$
0	0	1	0	1	1
0	1	0	0	0	$0 \rightarrow M_2$
0	1	1	0	1	1
1	0	0	0	0	$0 \rightarrow M_4$
1	0	1	0	0	$0 \rightarrow M_5$
1	1	0	1	0	1
1	1	1	1	0	1

F = 0 at rows 0, 2, 4, 5 → confirms $\Pi M(0, 2, 4, 5)$ ✓

Question 5 — Boolean Expression Simplification

 **Topic: Boolean Algebra Simplification (with De Morgan's)**

Q5. Simplify the Boolean expression: $((\bar{B}\bar{C}\bar{D} + ABC) \cdot D) + \bar{A}D$
(Top bar over the whole bracket means complement of the inner expression)

Given Expression

$$F = [(\bar{B}\bar{C}\bar{D} + ABC) \cdot D]' + \bar{A}D$$

(Note: The overbar covers the entire $(\bar{B}\bar{C}\bar{D} + ABC)D$ part)

Step 1: Apply De Morgan's Theorem

$$(X \cdot Y)' = X' + Y'$$

Let $X = (\bar{B}\bar{C}\bar{D} + ABC)$ and $Y = D$.

$$F = (\bar{B}\bar{C}\bar{D} + ABC)' + D' + \bar{A}D$$

Step 2: Apply De Morgan's to $(\bar{B}\bar{C}\bar{D} + ABC)'$

$$(P + Q)' = P' \cdot Q'$$

$$= (\bar{B}\bar{C}\bar{D})' \cdot (ABC)'$$

$$= (B + C + D) \cdot (A' + B' + C')$$

Step 3: Substitute back

$$F = (B + C + D)(A' + B' + C') + D' + \bar{A}D$$

Step 4: Simplify $D' + \bar{A}D$

Using consensus: $D' + \bar{A}D = D' + \bar{A}$ (since $X' + XY = X' + Y$)

$$F = (B + C + D)(A' + B' + C') + D' + \bar{A}$$

Step 5: Expand the product (using distributive)

$$(B + C + D)(A' + B' + C')$$

$$= BA' + BB' + BC' + CA' + CB' + CC' + DA' + DB' + DC'$$

$$= A'B + 0 + BC' + A'C + B'C + 0 + A'D + B'D + C'D$$

$$= A'B + BC' + A'C + B'C + A'D + B'D + C'D$$

Step 6: Add the remaining terms ($D' + A'$)

$$F = A'B + BC' + A'C + B'C + A'D + B'D + C'D + D' + A'$$

Step 7: Apply Absorption ($X + XY = X$)

- A' absorbs $A'B$, $A'C$, $A'D \rightarrow$ leaves A'
- D' absorbs (nothing here, D' stands alone)

$$F = A' + BC' + B'C + B'D + C'D + D'$$

Step 8: Further simplification

$$D' + C'D = D' + C' \text{ (consensus: } X' + X \cdot Y = X' + Y)$$

$$D' + B'D = D' + B' \text{ (same rule)}$$

$$F = A' + BC' + B'C + B' + C' + D'$$

$$B' + B'C = B' \text{ (absorption)}$$

$$C' + BC' = C' \text{ (absorption)}$$

$$F = A' + B' + C' + D' + B'C$$

$$B' + B'C = B' \text{ (absorption again)}$$

$$\mathbf{F = A' + B' + C' + D'}$$

Final Simplified Expression

By De Morgan's theorem: $A' + B' + C' + D' = (ABCD)'$

$$\mathbf{F = (ABCD)' = \text{NAND}(A, B, C, D)}$$

This means the function outputs 0 only when all inputs are 1 (i.e., when $ABCD = 1111$), otherwise outputs 1 — equivalent to a 4-input NAND gate.

Question 6 — Modify Logic Diagram (OR ↔ AND swap)

Topic: Logic Diagram Analysis & Modification

Q6. Change each OR to an AND, and each AND to an OR, in the figure. Then write the expression for the output X.

Original Circuit Description

Inputs: A, B, C (passing through NOT gates → A', B', C') and D directly.

Three AND gates feed into one OR gate to produce X.

Original AND gates:

- AND-1: inputs A', B → output = A'B
- AND-2: inputs B', C → output = B'C
- AND-3: inputs C', D → output = C'D

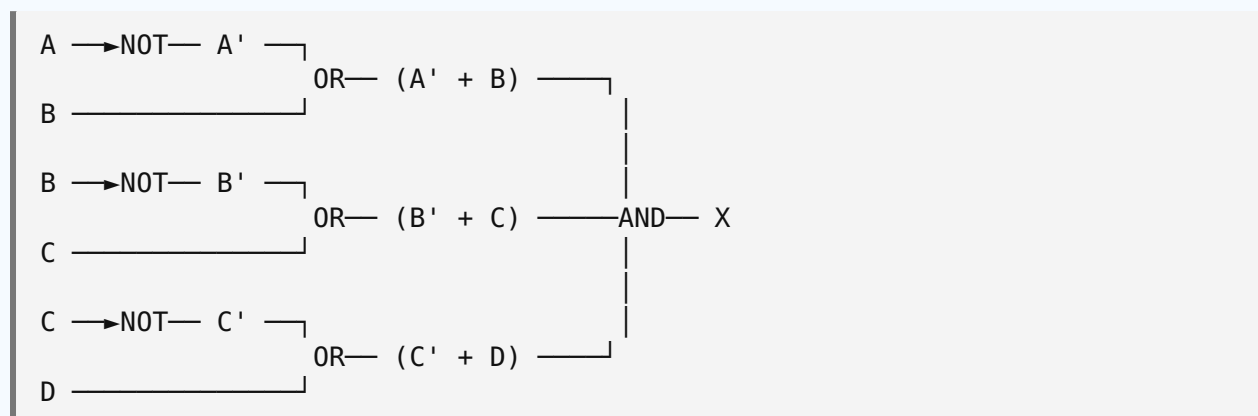
Original OR gate: $X = A'B + B'C + C'D$

After Modification (each AND → OR, each OR → AND)

The same circuit topology, but gates are swapped:

- OR-1 (was AND): $A' + B$
- OR-2 (was AND): $B' + C$
- OR-3 (was AND): $C' + D$
- Final AND gate (was OR): combines all three

Modified Circuit Diagram



Output Expression

$$X = (A' + B) \cdot (B' + C) \cdot (C' + D)$$

Observation (Duality Principle)

The new expression is the **dual** of the original. By the principle of duality, swapping AND ↔ OR (and 0 ↔ 1) in any Boolean expression produces its dual — a fundamental property in Boolean algebra.

Original	Modified (Dual)
$X = A'B + B'C + C'D$	$X = (A'+B)(B'+C)(C'+D)$

Question 7 — Express F as Product of Maxterms

 Topic: SOP to Canonical POS Conversion

Q7. Express the Boolean function $F(A, B, C, D) = ABC + A'B + AD'$ as a product of maxterms.

Step 1: Build the Truth Table

m	A	B	C	D	ABC	A'B	AD'	F
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	0
2	0	0	1	0	0	0	0	0
3	0	0	1	1	0	0	0	0
4	0	1	0	0	0	1	0	1
5	0	1	0	1	0	1	0	1
6	0	1	1	0	0	1	0	1
7	0	1	1	1	0	1	0	1
8	1	0	0	0	0	0	1	1
9	1	0	0	1	0	0	0	0
10	1	0	1	0	0	0	1	1
11	1	0	1	1	0	0	0	0
12	1	1	0	0	0	0	1	1
13	1	1	0	1	0	0	0	0
14	1	1	1	0	1	0	1	1
15	1	1	1	1	1	0	0	1

Step 2: Identify Minterms (where $F = 1$) and Maxterms (where $F = 0$)

$F = 1$ at minterms: 4, 5, 6, 7, 8, 10, 12, 14, 15

$F = 0$ at maxterms: 0, 1, 2, 3, 9, 11, 13

Step 3: Write Product of Maxterms

$$F(A, B, C, D) = \Pi M(0, 1, 2, 3, 9, 11, 13)$$

Step 4: Expanded Form

Maxterm	A B C D	Expression
---------	---------	------------

M_0	0 0 0 0	$(A + B + C + D)$
M_1	0 0 0 1	$(A + B + C + D')$
M_2	0 0 1 0	$(A + B + C' + D)$
M_3	0 0 1 1	$(A + B + C' + D')$
M_9	1 0 0 1	$(A' + B + C + D')$
M_{11}	1 0 1 1	$(A' + B + C' + D')$
M_{13}	1 1 0 1	$(A' + B' + C + D')$

$$F = (A+B+C+D)(A+B+C+D')(A+B+C'+D)(A+B+C'+D')(A'+B+C+D')(A'+B+C'+D')(A'+B'+C+D')$$

Question 8 — Home Security System Design

Topic: Combinational Logic Design from Word Problem

Q8. Home security system has three sensors: door sensor (A), motion detector (B), arm/disarm switch (C).

The alarm activates when:

- System is armed AND door is open, OR
- System is disarmed AND motion detected AND door is open, OR
- System is armed AND no motion detected AND door is closed.

Find Boolean function $P(A, C, B)$ and construct the logic diagram.

Step 1: Define Variables

Variable	Meaning	Logic 1	Logic 0
A	Door sensor	Door open	Door closed
B	Motion detector	Motion detected	No motion
C	Arm/disarm switch	Armed	Disarmed

Step 2: Translate Each Condition to Boolean

Condition	Boolean Term
Armed AND door open	$C \cdot A = AC$
Disarmed AND motion AND door open	$C' \cdot B \cdot A = ABC'$
Armed AND no motion AND door closed	$C \cdot B' \cdot A' = A'B'C$

Step 3: Boolean Function

$$P(A, B, C) = AC + ABC' + A'B'C$$

Step 4: Truth Table

m	A	B	C	AC	ABC'	A'B'C	P
0	0	0	0	0	0	0	0
1	0	0	1	0	0	1	1
2	0	1	0	0	0	0	0
3	0	1	1	0	0	0	0
4	1	0	0	0	0	0	0
5	1	0	1	1	0	0	1

6	1	1	0	0	1	0	1
7	1	1	1	1	0	0	1

$$P = \Sigma m(1, 5, 6, 7)$$

Step 5: Simplify Using K-Map

A \ BC	00	01	11	10
0	0	1	0	0
1	0	1	1	1

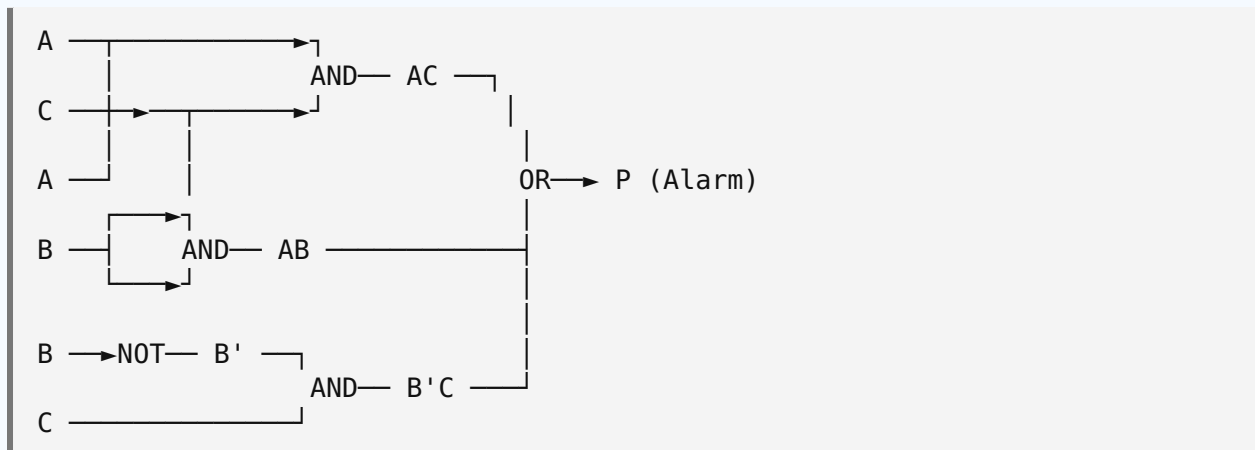
Groupings:

- Pair {m1, m5}: column 01 $\rightarrow$ BC' is 0, so this is $C \cdot B' \rightarrow \mathbf{B'C}$
- Pair {m5, m7}: row A=1, column changes B $\rightarrow \mathbf{AC}$
- Pair {m6, m7}: row A=1, column 11 and 10 $\rightarrow \mathbf{AB}$

Step 6: Simplified Expression

$$\mathbf{P = AC + AB + B'C}$$

Step 7: Logic Diagram



Gates required: 1 NOT, 3 AND (2-input), 1 OR (3-input).

Question 9 — K-Map Simplification with Don't Cares

Topic: K-Map (4-variable) with Don't-Care Conditions

Q9. Using K-map, simplify $F(A, B, C, D) = \Sigma m(0, 1, 3, 5, 6, 7, 12, 14) + \Sigma d(2, 8, 11, 15)$. Draw the logic diagram.

Step 1: Construct the K-Map (4 variables)

Rows: AB (00, 01, 11, 10) | Columns: CD (00, 01, 11, 10)

AB \ CD	00	01	11	10
00	1 (0)	1 (1)	1 (3)	× (2)
01	0 (4)	1 (5)	1 (7)	1 (6)
11	1 (12)	0 (13)	× (15)	1 (14)
10	× (8)	0 (9)	× (11)	0 (10)

Step 2: Identify Optimal Groupings

Group 1 (Octet/Quad of 4): Cells {0, 1, 3, 2(d)} and {5, 7, ...} — let's pick the columns CD=01 and CD=11 for rows AB=00, 01:

Cells {1, 3, 5, 7} → AB=00,01 and CD=01,11 → A is 0, D is 1 → **A'D**

Group 2 (Quad of 4): Cells {0, 1, 2(d), 3} → row AB=00 → **A'B'**

Group 3 (Quad of 4): Cells {6, 7, 14, 15(d)} → CD=11 and 10, AB=01 and 11 → **BC**

Group 4 (Quad of 4): Cells {12, 14, 8(d), ...} — let's check {12, 14}: row AB=11, columns 00 and 10. Pair only = ABD'.

Better: {12, 14, 8(d), 10(=0 NO)}. So cells {12,14} only with d8: {8, 12} = AC'D' (A=1, C=0, D=0).

Combine {8, 12} (column 00, AB=10,11) with {14}: cells {8, 12, 10(=0), 14} → can't use 10.

Take pair {12, 14}: AB=11, D=0 → **ABD'**

Step 3: Final Simplified Expression

$$F = A'B' + A'D + BC + ABD'$$

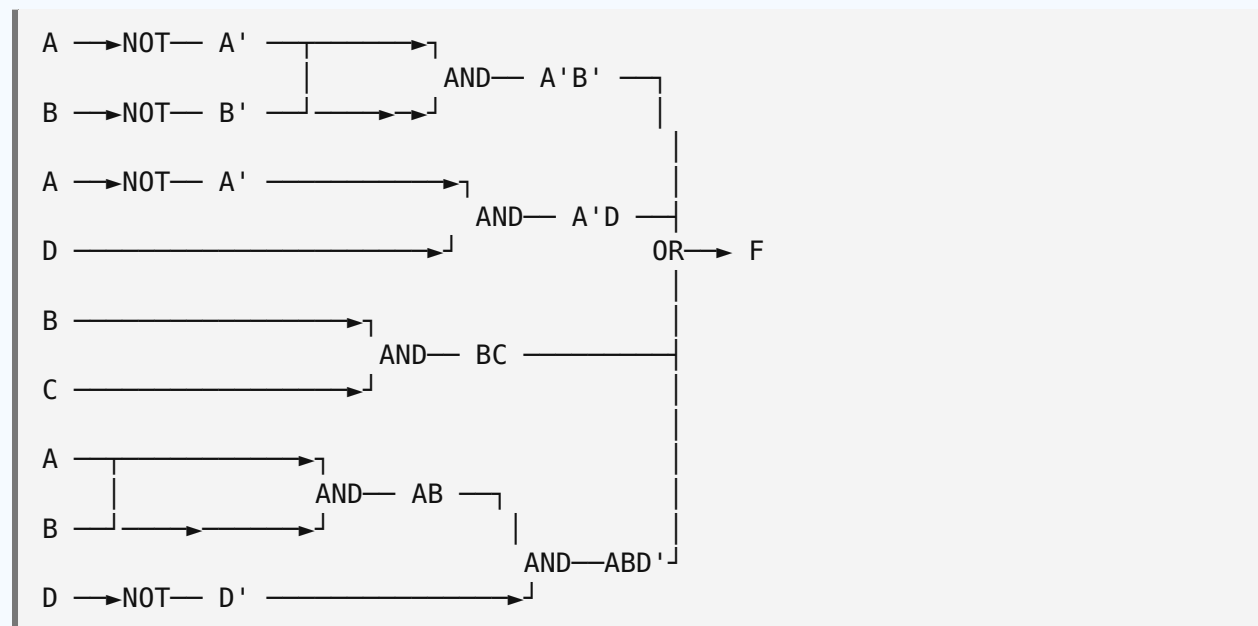
Alternative equivalent form (using different grouping):

$F = A'B' + A'D + BC + ABD'$ (same result)

Step 4: Verify with Truth Table (Spot Check)

- m0 (0000): $A'B'=1$ ✓
- m5 (0101): $A'D=1$ ✓
- m7 (0111): $A'D=1, BC=1$ ✓
- m12 (1100): $ABD'=1$ ✓
- m14 (1110): $BC=1, ABD'=1$ ✓

Step 5: Logic Diagram



Gates required: 3 NOT (A' , B' , D'), 4 AND, 1 OR (4-input).

Question 10 — Working Principle of All Flip-Flops

📌 Topic: Sequential Logic — Flip-Flops

Q10. Explain the working principle of every type of flip-flop in digital electronics, with necessary figures and tables.

What is a Flip-Flop?

A flip-flop is a **1-bit memory element** — a sequential logic device whose output depends on both current inputs and previous state. They are the basic building blocks of registers, counters, and memory.

There are **four main types** of flip-flops: SR, JK, D, and T.

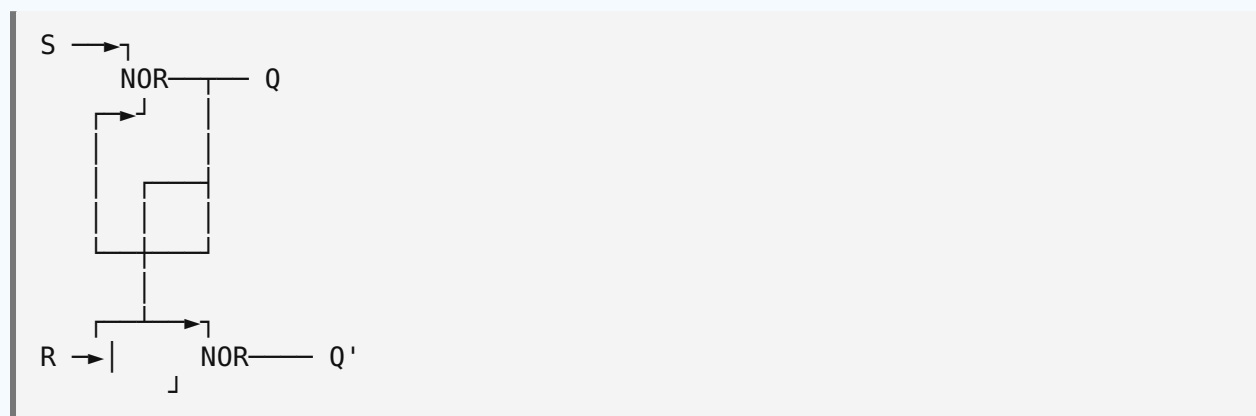
1. SR Flip-Flop (Set-Reset)

The simplest flip-flop. Two inputs: **S (Set)** and **R (Reset)**, plus a clock. Outputs: Q and Q'.

Truth Table (Characteristic Table):

S	R	Q(t+1)	State
0	0	Q(t)	No change
0	1	0	Reset
1	0	1	Set
1	1	?	Invalid (forbidden)

Characteristic Equation: $Q(t+1) = S + R' \cdot Q(t)$, with constraint $S \cdot R = 0$. **Logic Diagram (using NOR gates):**



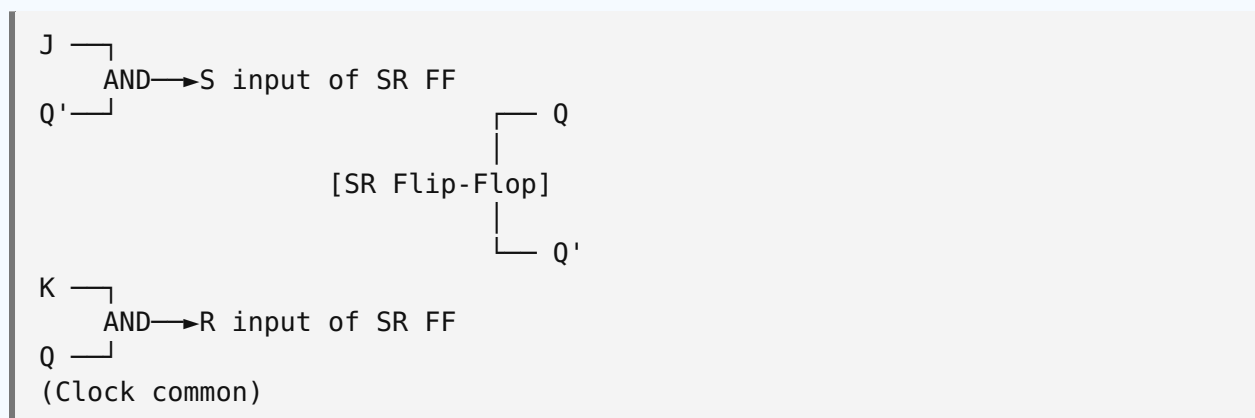
2. JK Flip-Flop

Improved version of SR — eliminates the invalid state. J replaces S, K replaces R, and $J=K=1$ causes the output to **toggle**.

Truth Table:

J	K	$Q(t+1)$	State
0	0	$Q(t)$	No change
0	1	0	Reset
1	0	1	Set
1	1	$Q'(t)$	Toggle

Characteristic Equation: $Q(t+1) = JQ' + K'Q$ **Logic Diagram:**



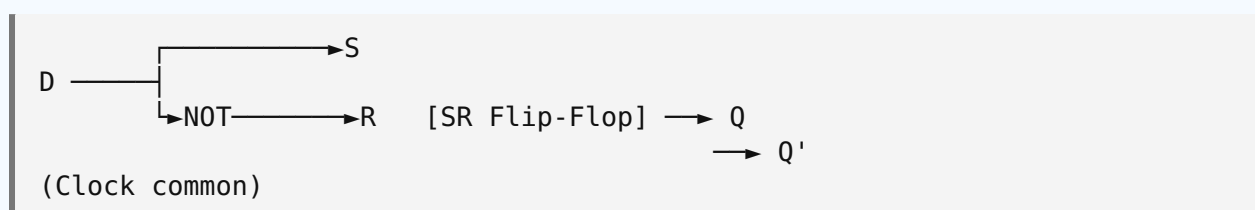
3. D Flip-Flop (Data / Delay)

Has only **one data input D**. Output Q simply follows D on the clock edge — used widely for data storage.

Truth Table:

D	$Q(t+1)$	State
0	0	Reset
1	1	Set

Characteristic Equation: $Q(t+1) = D$ **Logic Diagram (built from SR):**



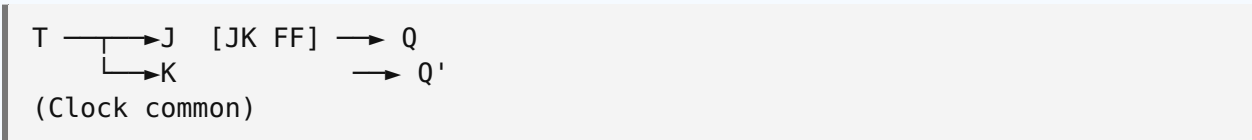
4. T Flip-Flop (Toggle)

Has **one input T**. When T=0, Q stays the same. When T=1, Q toggles. Used in counters.

Truth Table:

T	Q(t+1)	State
0	Q(t)	No change
1	Q'(t)	Toggle

Characteristic Equation: $Q(t+1) = T \oplus Q(t) = TQ' + T'Q$ Logic Diagram (built from JK with J = K = T):



📌 Summary Comparison Table

Type	Inputs	Characteristic Equation	Main Use
SR	S, R	$Q(t+1) = S + R'Q$	Basic memory
JK	J, K	$Q(t+1) = JQ' + K'Q$	General purpose; counters
D	D	$Q(t+1) = D$	Data storage; registers
T	T	$Q(t+1) = T \oplus Q$	Counters; frequency dividers

Excitation Tables (for design)

Q → Q'	SR	JK	D	T
0 → 0	0×	0×	0	0
0 → 1	10	1×	1	1
1 → 0	01	×1	0	1
1 → 1	×0	×0	1	0

Question 12 — Full Subtractor using Half Subtractors

 **Topic: Combinational Logic Circuits — Subtractors**

Q12. Design a Full Subtractor circuit using Half Subtractors and necessary basic gates. Show the relevant output equations.

Step 1: Recall — Half Subtractor

Computes $A - B$ for two single bits.

A	B	D (Difference)	B _{out}
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

$$D = A \oplus B$$

$$B_{\text{out}} = A' \cdot B$$

Step 2: Full Subtractor — Truth Table

Inputs: A (minuend), B (subtrahend), B_{in} (incoming borrow). Outputs: D and B_{out}.

A	B	B _{in}	D	B _{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Step 3: Boolean Equations

$$D = \Sigma m(1, 2, 4, 7) = A \oplus B \oplus B_{in}$$

$$B_{out} = \Sigma m(1, 2, 3, 7) = A'B + A'B_{in} + BB_{in} = A'B + (A \oplus B)' \cdot B_{in}$$

For the half-subtractor design, the simpler form is:

$$D = (A \oplus B) \oplus B_{in}$$

$$B_{out} = A'(B \oplus B_{in})' \cdot \dots \rightarrow \text{simpler: } B_{out} = A'B + (A \oplus B)' \cdot B_{in}$$

$$\text{Or equivalently: } B_{out} = (\text{HS1 borrow}) + (\text{HS2 borrow})$$

Step 4: Building Full Subtractor with Two Half Subtractors

Stage 1 — HS1:

Inputs: A, B

Outputs: $D_1 = A \oplus B$; $b_1 = A'B$

Stage 2 — HS2:

Inputs: D_1 , B_{in}

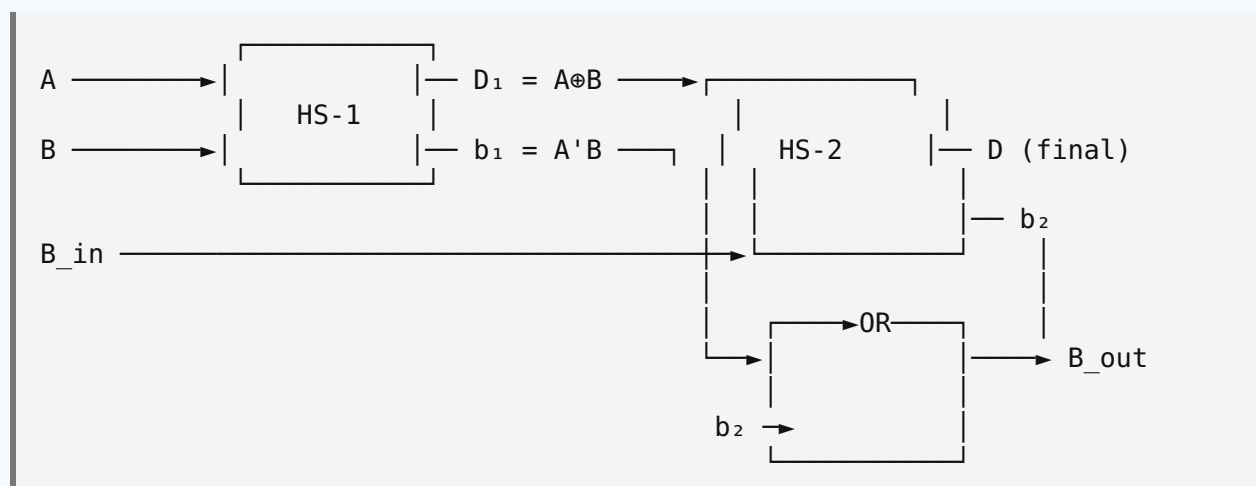
Outputs: $D = D_1 \oplus B_{in} = A \oplus B \oplus B_{in}$ (final difference)

$$b_2 = D_1' \cdot B_{in} = (A \oplus B)' \cdot B_{in}$$

Combine borrows with an OR gate:

$$B_{out} = b_1 + b_2 = A'B + (A \oplus B)' \cdot B_{in}$$

Step 5: Final Block Diagram



Summary of Equations

Output	Equation
Difference (D)	$D = A \oplus B \oplus B_{in}$

Borrow out (B_{out})

$$B_{\text{out}} = A'B + (A \oplus B)' \cdot B_{\text{in}}$$

Components used: 2 Half Subtractors + 1 OR gate.

Question 13 — 4-input Priority Encoder (Custom Priority)

Topic: Priority Encoder Design

Q13. Design a 4-input priority encoder with inputs D_0, D_1, D_2, D_3 , where:
 D_1 = highest priority, then D_0 , then D_2 , and D_3 = lowest priority.

Step 1: Priority Order

Priority	Input	Output Code (xy)
1st (highest)	D_1	00
2nd	D_0	01
3rd	D_2	10
4th (lowest)	D_3	11

Output: x, y (2-bit binary code) and V (Valid: 1 if any input is active).

Step 2: Truth Table (Priority Logic)

When a higher-priority input is 1, lower-priority inputs are "don't care" ($\times$).

D_1	D_0	D_2	D_3	x	y	V
0	0	0	0	$\times$	$\times$	0
0	0	0	1	1	1	1
0	0	1	$\times$	1	0	1
0	1	$\times$	$\times$	0	1	1
1	$\times$	$\times$	$\times$	0	0	1

Step 3: Derive Boolean Expressions

Output $x = 1$ when the active highest-priority input is D_2 or D_3 .

From the table: $x = 1$ when $D_1=0, D_0=0$, AND ($D_2=1$ OR ($D_2=0, D_3=1$))

$$= D_1' \cdot D_0' \cdot D_2 + D_1' \cdot D_0' \cdot D_2' \cdot D_3$$

$$= D_1' \cdot D_0' \cdot (D_2 + D_2' \cdot D_3)$$

$$= D_1' \cdot D_0' \cdot (D_2 + D_3)$$

$$x = D_1' \cdot D_0' \cdot (D_2 + D_3)$$

Output $y = 1$ when the active highest-priority input is D_0 or D_3 .

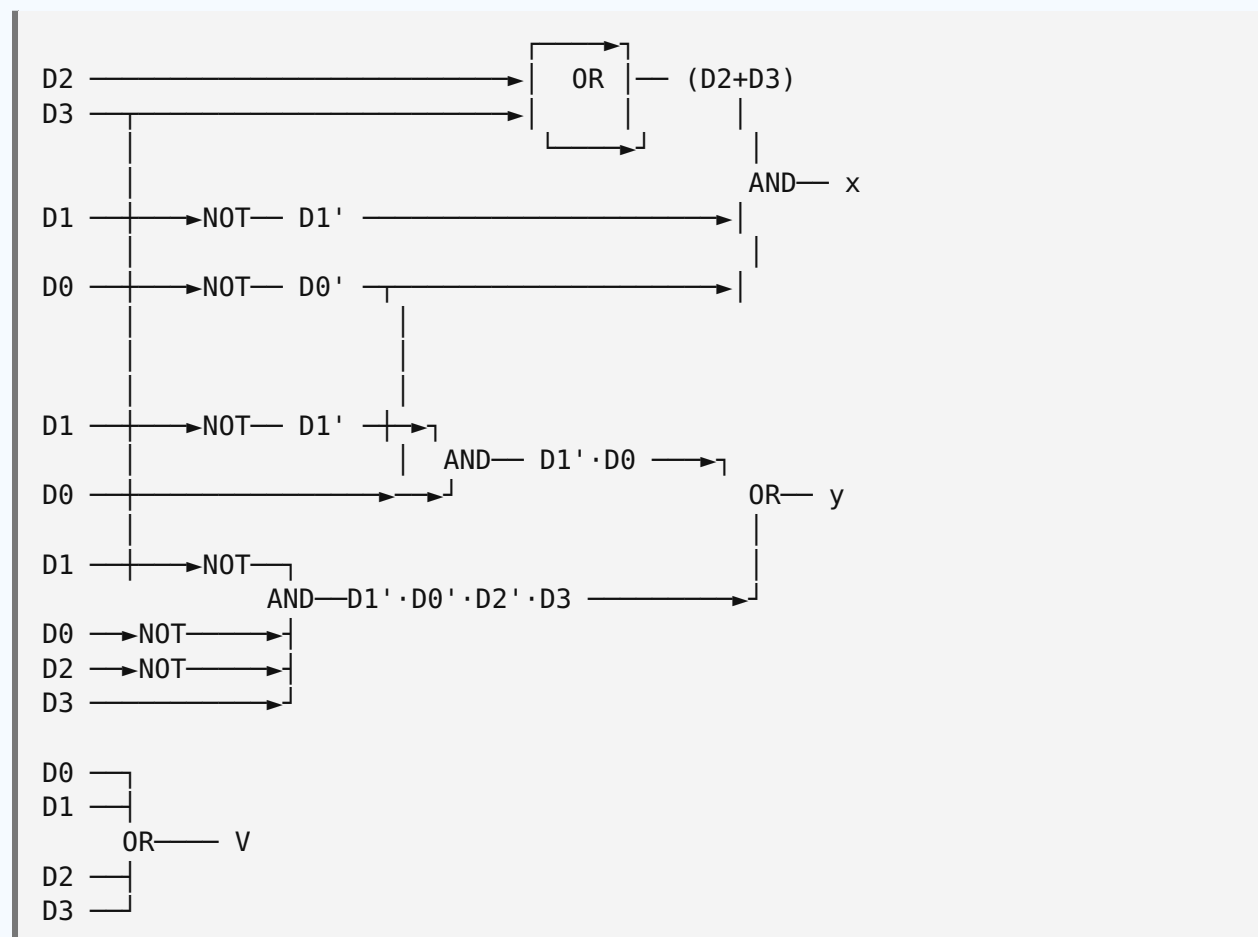
$$= D_1' \cdot D_0 + D_1' \cdot D_0' \cdot D_2' \cdot D_3$$

$$y = D_1' \cdot D_0 + D_1' \cdot D_0' \cdot D_2' \cdot D_3$$

Valid (V) = 1 if any input is active.

$$V = D_0 + D_1 + D_2 + D_3$$

Step 4: Logic Diagram



Final Equations Summary

Output	Equation
x	$D_1' \cdot D_0' \cdot (D_2 + D_3)$
y	$D_1' \cdot D_0 + D_1' \cdot D_0' \cdot D_2' \cdot D_3$

V	$D_0 + D_1 + D_2 + D_3$
---	-------------------------



Assignment Summary — All Topics & Final Answers

Q#	Topic	Final Answer
1	Number System	Binary: 1111111111111111; Decimal: 65535; Hex: FFFF
2(a)	r's Complement	16's complement = $(B86D)_{16}$
2(b)	Hex $\rightarrow$ Binary	$(1011\ 1000\ 0110\ 1101)_2$
2(c)	Binary $\rightarrow$ Octal	$(134155)_8$
3	Subtraction by 2's Comp.	$(11)_{10} = (00001011)_2$
4	SOP $\rightarrow$ POS	$F = \Pi M(0,2,4,5)$
5	Boolean Simplification	$F = A' + B' + C' + D' = (ABCD)'$
6	Logic Diagram (dual)	$X = (A'+B)(B'+C)(C'+D)$
7	SOP $\rightarrow$ POS (4-var)	$F = \Pi M(0,1,2,3,9,11,13)$
8	Combinational Design	$P = AC + AB + B'C$
9	K-Map with don't-cares	$F = A'B' + A'D + BC + ABD'$
10	Flip-Flops	SR, JK, D, T (with eqns and tables)
12	Full Subtractor from HS	$D = A \oplus B \oplus B_{in}; B_{out} = A'B + (A \oplus B)' \cdot B_{in}$
13	Priority Encoder	$x = D_1'D_0'(D_2+D_3); y = D_1'D_0 + D_1'D_0'D_2'D_3; V = D_0+D_1+D_2+D_3$

DLD Assignment — Course Code: CSE 211

Metropolitan University, Department of Computer Science and Engineering

All 12 questions solved with topic mapping. Best wishes! 🎓